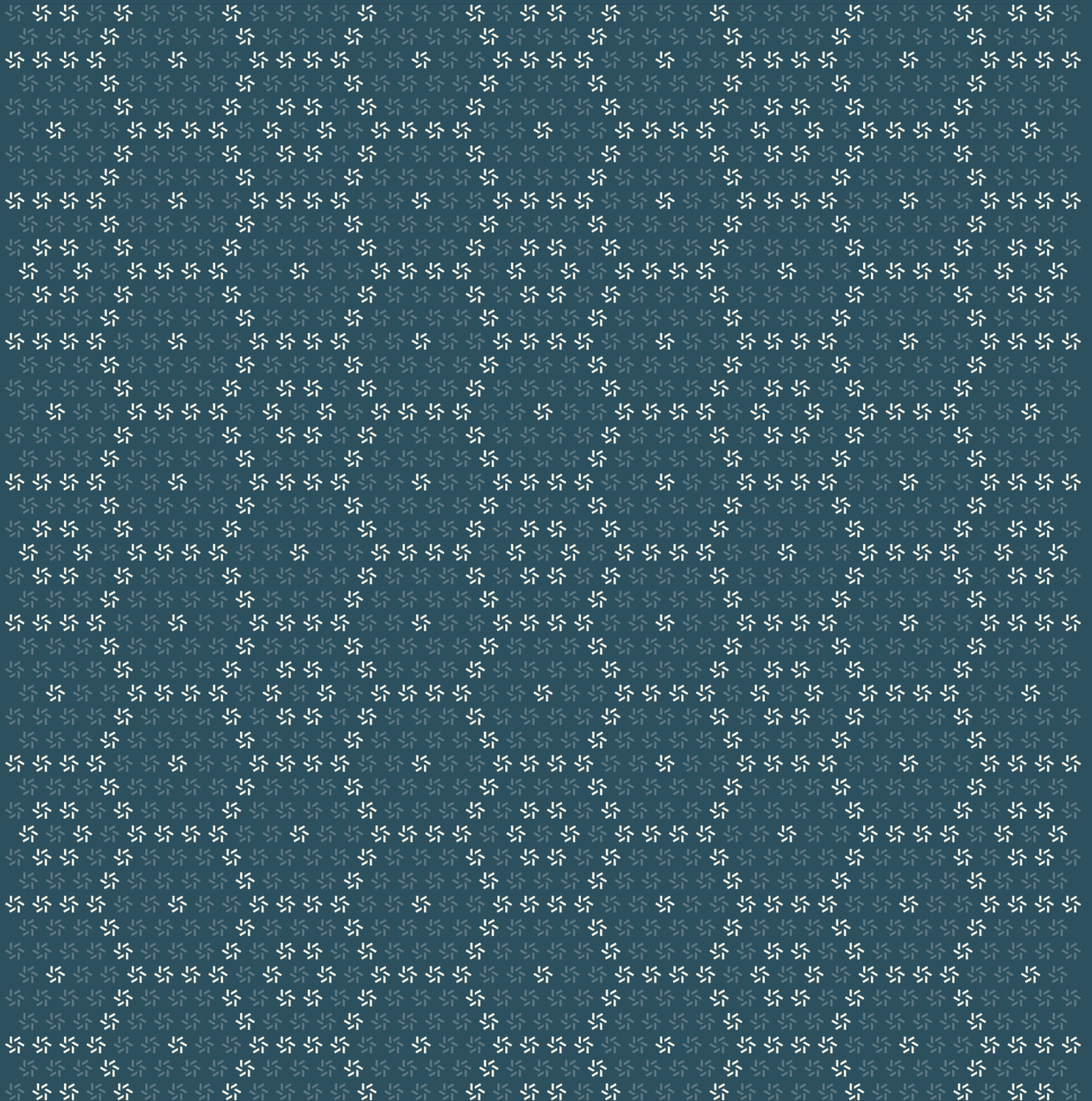


July 15, 2026

Zcash Ironwood Upgrade

Cryptographic Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	7
2.1. About Zcash Ironwood Upgrade	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	11
3.1. <code>rcm_v3</code> relies on implementation-specific behavior of <code>Fp::to_repr</code>	11
3.2. <code>Bundle::commitment</code> accepts incompatible pool restrictions for V6 transactions	13
3.3. Check <code>circuit_version</code> of verifying key against bundle	15
4. Discussion	17
4.1. ZIP 2005 uses an inconsistent API for $H^{\text{rcm}, \text{Orchard}}$ in the Recovery Statement	17

5.	Design	18
5.1.	Cross-Address Restriction in Orchard and Ironwood Bundles	18
5.2.	Version 3 Note Commitment Trapdoor (rcm)	20

6.	Assessment Results	22
6.1.	Disclaimer	22

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [↗] worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [↗] and follow [@zellic_io](#) [↗] on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io [↗].



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Valar Group from June 26th to July 10th, 2026. During this engagement, Zellic reviewed Zcash Ironwood Upgrade's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is the newly introduced `enableCrossAddress` flag in Orchard and Ironwood bundles implemented as specified?
 - Is the derivation of the trapdoor of a version 3 note commitment implemented as specified?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Whether the specified design is suitable for the intended goals, in particular enabling post-quantum recoverability
- Correctness of the code at the base commit of the audited diff, in particular all parts of the Orchard action circuit besides the cross-address restriction
- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

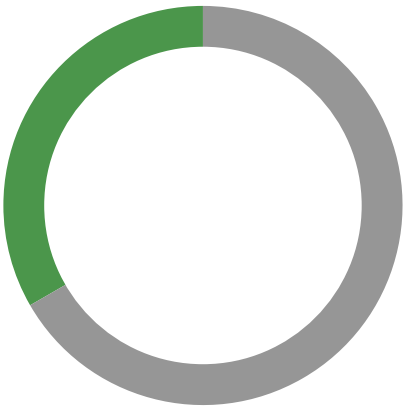
During our assessment on the scoped Zcash Ironwood Upgrade targets, we discovered three findings. No critical issues were found. One finding was of low impact and the other findings were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Valar

Group in the Discussion section ([4.](#), [7.](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	1
Informational	2



2. Introduction

2.1. About Zcash Ironwood Upgrade

Valar Group contributed the following description of Zcash Ironwood Upgrade:

Orchard is the latest shielded pool for Zcash, using the Halo2 proof system. Recently there was a critical security bug found, leading to the NU6.2 hard fork. This audit was prepared to look for any further bugs, and to review the changes to the circuit for the incident response. Which was:

- Updating the circuit to add a new "enable cross address flag", which lets users split up their funds to themselves, while limiting the usage of regular payments. (Multi-signer payment flows are allowed)
- Adding the new Ironwood pool, which uses the same circuit as Orchard
- Adding the v6 tx format's signing for Ironwood
- The integration of quantum recoverability

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Protocol design. For novel/nonstandard protocols, we investigate the core design for potential flaws. This does not, however, apply to implementations of established protocols and algorithms. For example, when auditing a library for Yao's garbled circuits, we would not spend time on the garbling scheme itself. On the other hand, the white paper for a novel protocol would be very much considered in scope and examined thoroughly.

Applied cryptography. Zellic has a dedicated team of strong theoretical and applied cryptographers. Implementing cryptographic applications securely, like Web3 wallets, is incredibly difficult, and we help clients navigate a minefield of potential pitfalls and mistakes. We have ensured secure implementation of noncustodial wallets, ERC-4337 (AA), MPC, Shamir's secret sharing (SSS), EOAs, native multi-sig support, enclave solutions, social log-in, and key recovery, among other technologies.

Cryptographic components. We manually review the cryptography usage of the project and examine the relevant studies and standards for any inconsistencies or vulnerabilities. We ensure all of the cryptographic primitives used in the project are used and configured correctly.

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple,

surface-level mistakes that could have easily been caught ahead of time by code review. For cryptographic applications, this includes insufficient checks on the inputs, incorrect handling of edge cases, and completeness issues in low-level arithmetic operations.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped targets itself. These observations — found in the Discussion (4.7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Zcash Ironwood Upgrade Targets

Type	Rust
Platform	Zcash
Target	Only changes between 82e0739c...30c4ea27
Repository	https://github.com/zcash/orchard ↗
Version	30c4ea2788375c542bfaa4704a3a772308729321
Programs	address.rs bundle.rs bundle/batch.rs circuit.rs note.rs

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 2.9 person-weeks. The assessment was conducted by two consultants over the course of two calendar weeks and one calendar day.

Contact Information

The following project manager was associated with the engagement:

Chad McDonald
✂ Engagement Manager
chad@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Mark Pedron
✂ Engineer
mark.pedron@zellic.io ↗

Mohit Sharma
✂ Engineer
mohit@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

June 26, 2026 Start of primary review period

July 10, 2026 End of primary review period

3. Detailed Findings

3.1. rcm_v3 relies on implementation-specific behavior of `Fp::to_repr`

Target	orchard::note		
Category	Cryptography	Severity	High
Likelihood	Low	Impact	Low

Description

The derivation of the note-version 3 (ZIP 2005, quantum-recoverable) rcm is implemented in `RandomSeed::rcm_v3` in `src/note.rs`. The `rho` and `psi` field elements are serialized into the BLAKE2b PRF input by calling `to_repr()` directly on the underlying `pallas::Base` values:

```
// src/note.rs
pub(crate) fn rcm_v3(
    // ...
) -> commitment::NoteCommitTrapdoor {
    let mut h = Blake2bParams::new()
        .hash_length(64)
        .personal(PRF_EXPAND_PERSONALIZATION)
        .to_state();
    // ...
    // rho: LEBS20SP_256(repr_P(rho)) - Pallas base field canonical repr (32 bytes)
    h.update(&rho.0.to_repr());
    // psi: LEBS20SP_256(repr_P(psi)) - Pallas base field canonical repr (32 bytes)
    h.update(&psi.to_repr());
    // ...
}
```

Both update statements depend on the concrete bytes returned by `to_repr()`. This function is part of the `PrimeField` trait from the `ff` crate, whose documentation explicitly states that the byte layout is not part of the API contract:

```
// ff-0.13.0/src/lib.rs
/// Converts an element of the prime field into the standard byte
/// representation for
/// this field.
///
/// The endianness of the byte representation is implementation-specific.
/// Generic
```

```
/// encodings of field elements should be treated as opaque.  
fn to_repr(&self) -> Self::Repr;
```

The `pasta_curves` crate that provides the `pallas::Base` implementation does not add any deviating guarantee about the stability of this byte representation on `to_repr`. Although the current implementation happens to produce the canonical little-endian encoding required by the specification, this is an implementation detail rather than a contractual guarantee, and it is not enforced by the trait.

Because `rcm_v3` is newly introduced code, it establishes a fresh dependency on this implementation-specific behavior.

Impact

The value of `rcm` is not enforced on chain, so a transaction that derives `rcm` from a divergent byte representation would still be accepted by the network. The divergence would only manifest later: the receiving party recomputes the note commitment and, after implementation of the post-quantum recovery plan defined by ZIP 2005, would reject the note. The funds would then become unspendable.

Unlike consensus-critical serializations such as the extracted note commitment (`cmx`) or nullifiers, a layout change here would not fail loudly at validation time. It would fail silently and only for the affected notes, making the resulting fund loss difficult to detect and diagnose.

Recommendations

Do not rely on the implicit byte layout of `to_repr()`. Instead, define the serialization of `rho` and `psi` explicitly at the point of use so that the encoding required by ZIP 2005 is fixed by this code and independent of any implementation-specific behavior of the underlying field type.

If an explicit, representation-independent serialization is not practical, at a minimum pin the encoding with a test vector that asserts a known `pallas::Base` value serializes to a fixed 32-byte constant, so that any future change to the upstream byte layout is caught in CI rather than silently producing incorrect `rcm` values.

Remediation

This issue has been acknowledged by Valar Group.

3.2. Bundle::commitment accepts incompatible pool restrictions for V6 transactions

Target	orchard::bundle		
Category	Coding Mistakes	Severity	Informational
Likelihood	Low	Impact	Informational

Description

The method `Bundle::commitment`, as per the comments, is intended to return an error if the pool restrictions and transaction version it is called with are incompatible. This check is implemented in `BundlePoolRestrictions::commitment_format` as follows:

```
impl BundlePoolRestrictions {
    fn commitment_format(
        self,
        tx_version: TxVersion,
    ) -> Result<BundleCommitmentFormat, CommitmentError> {
        Ok(match self {
            BundlePoolRestrictions::OrchardPreNu6_2
            | BundlePoolRestrictions::OrchardNu6_20only
            | BundlePoolRestrictions::OrchardNu6_30nward => match tx_version {
                TxVersion::V5 => BundleCommitmentFormat::OrchardV5,
                TxVersion::V6 => BundleCommitmentFormat::OrchardV6,
            },
            BundlePoolRestrictions::IronwoodNu6_30nward => match tx_version {
                TxVersion::V5 =>
                return Err(CommitmentError::InvalidTransactionVersion),
                TxVersion::V6 => BundleCommitmentFormat::IronwoodV6,
            },
        })
    }
}
```

As shown, `commitment_format` does not raise an error if a `BundlePoolRestrictions` struct of the type `OrchardPreNu6_2` or `OrchardNu6_20only` is passed alongside `TxVersion::V6`, even though these combinations are invalid by consensus.

Impact

The behavior does not lead to an on-chain bypass of restrictions. It may cause a client to continue constructing, proving, or signing a V6 transaction using Pre-NU6.3 bundle semantics, but the resulting transaction will be rejected on chain, so it only leads to a waste of client-side computation.

Recommendations

We recommend updating `BundlePoolRestrictions::commitment_format` to return `CommitmentError::InvalidTransactionVersion` in the case of `TxVersion::V6` coupled with Pre-NU6.3 restrictions. We also recommend expanding the test suite to cover these cases.

Remediation

This issue has been acknowledged by Valar Group.

3.3. Check circuit_version of verifying key against bundle

Target	orchard::circuit / orchard::bundle		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

In `orchard::circuit::Proof::verify`, the following code segment was added to prevent verification of a bundle with flag `enableCrossAddress=false` against a verifying key that does not support this check:

```
// src/circuit.rs
pub fn verify(&self, vk: &VerifyingKey, instances: &[Instance]) -> Result<(),
    plonk::Error> {
    if instances.iter().any(Instance::cross_address_disabled)
        && !vk.supports_cross_address_restriction()
    {
        return Err(plonk::Error::InvalidInstances);
    }
    // ...
}
```

Similarly, in the batch verifier, the following code was added:

```
// src/bundle/batch.rs
pub fn add_bundle<V: Copy + Into<i64>>(
    &mut self,
    bundle: &Bundle<Authorized, V>,
    sighash: [u8; 32],
) -> Result<(), BatchError> {
    if !bundle.flags().cross_address_enabled()
        && !self.vk.supports_cross_address_restriction()
    {
        return Err(BatchError::RestrictionUnsupportedByKey);
    }
    // ...
}
```

This check is overly specific, because the caller has to ensure anyway that the correct verifying key is used. If this check is intended to prevent using the incorrect verifying key, it should be expanded to check the circuit version.

Impact

The current check does not completely prevent mixing up verifying keys. Using the incorrect key could result in an unsound check (if the Pre-NU6.2 key were used) or in consensus deviations (because NU6.3 requires that the correct verifying key is used for Ironwood bundles even if `enableCrossAddress` is set to `true`).

Recommendations

Check that the correct key is used by including the `OrchardCircuitVersion` derived from `pool_restrictions` in the `Bundle` struct. During bundle parsing, `pool_restrictions` is already required to parse the flags, so one could remember the value. Then, instead of checking for the specific `supports_cross_address_restriction` feature, one could check here that the `OrchardCircuitVersion` of the `Bundle`'s `pool_restrictions` and of the `VerifyingKey` agree.

Remediation

This issue has been acknowledged by Valar Group.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. ZIP 2005 uses an inconsistent API for $H^{\text{rcm}, \text{Orchard}}$ in the Recovery Statement

ZIP 2005 defines the new quantum-recoverable Orchard note commitment trapdoor derivation as a keyed hash over the actual note fields. It imports $H^{\text{rcm}, \text{Orchard}}$ from Orchard and uses the following definition:

$$H_{\text{rseed}}^{\text{rcm}, \text{Orchard}}(g^*_d, pk^*_d, v, \rho, \psi) = \text{ToScalar}^{\text{Orchard}}(\text{PRF}_{\text{rseed}}^{\text{expand}}(\text{pre_rcm}))$$

The declared API definition takes the note plaintext fields $g^*_d, pk^*_d, v, \rho, \psi$ directly as inputs, which are then used in the computation of pre_rcm .

Under this definition, the domain separator for the new rcm derivation is the leading byte 0x0B inside pre_rcm . The note plaintext lead byte 0x03 is not an argument to $H^{\text{rcm}, \text{Orchard}}$. Later, however, in the section of the spec titled "Proposed Recovery Statement", the same definition is imported but called with a different function signature:

$$\text{rcm} = H_{\text{rseed}}^{\text{rcm}, \text{Orchard}}(0x03, \text{noterepr})$$

where noterepr is a precomputed note representation. This call does not follow the definition's API. We note that at the implementation level, the rcm computation happens in rcm_v3 in note.rs and follows the originally imported definition, so this is merely a notation mismatch in the documentation and not a code-level issue.

5. Design

This section provides a detailed description of the audited code's design with regard to the audit's goals.

5.1. Cross-Address Restriction in Orchard and Ironwood Bundles

The Zcash NU6.3 upgrade is planned to introduce a mechanism to restrict actions in Orchard and Ironwood bundles to have the same address in the spend and output note. This restriction is then planned to be enforced in the same upgrade for all Orchard transactions to incentivize the migration to the Ironwood pool.

Note that this restriction does not prevent consenting parties from transferring between different addresses in the Orchard pool by incorporating multiple actions in the same bundle.

Specification

The specification [ZIP 229](#) defines the version 6 transaction format, which introduces the cross-address restriction flag. A version 6 transaction contains two `enableCrossAddress` flags, one for the Orchard and one for the Ironwood bundle. Each flag applies to all actions in its respective bundle.

ZIP 229 also refers to a consequence of the planned restriction on this flag in Orchard bundles:

However, NU6.3 enforces that outputs to the Orchard pool are sent to an Orchard-protocol-level address for which the transaction creator can authorize spends. The intent is that transfers between users will automatically create notes in the Ironwood pool.

During the audit, [ZIP 2006](#), which is planned to include this restriction, was not yet published and not considered. We therefore checked the correctness of the statement above against the implementation instead.

Implementation

The implementation is agnostic to the fact that transaction version 6 and disabling cross-address transfers in the Orchard pool happen at the same block height. The bundle struct is expanded to support Orchard bundles in both version 5 and 6 transactions and Ironwood bundles in version 6 transactions.

To create an `orchard::bundle::Bundle` struct, an `orchard::bundle::Flags` struct must be instantiated. When a bundle is parsed from a transaction, this must be done by calling the `orchard::bundle::Flags::from_byte` function. The parameter `pool_restrictions` of this function defines the allowed values and interpretation of the `enableCrossAddress` flag. It is the caller's duty to provide the correct value. If `pool_restrictions` is set to `BundlePoolRestrictions::OrchardNu6_30nward`, it is guaranteed that the field `cross_address_enabled` is false.

For a bundle constructed this way, there are two verifying paths that have to enforce the restriction:

Individual bundle

A call to

```
orchard::bundle::Bundle::verify_proof(&self,&orchard::circuit::VerifyingKey)
```

verifies the action circuit proofs for the public inputs using the `cross_address_enabled` values. It is the caller's duty to provide the correct `VerifyingKey`.

Batch

Use the `orchard::bundle::batch::BatchValidator` struct as follows:

- `::new(&orchard::circuit::VerifyingKey)` instantiates a new validator fixing the verifying key. It is the caller's duty to provide the correct `VerifyingKey`.
- `::add_bundle(&mut self, &Bundle, sighash)` can be called multiple times to add bundles.
- `::validate(self, rng)` validates the action circuit proofs as above and additionally verifies the authorization signatures.

We now describe how both functions guarantee the enforcement of the cross-address restriction. First, the existing Orchard action circuit is expanded to enforce $(g_{d_old}, pk_{d_old}) = (g_{d_new}, pk_{d_new})$ when the `cross_address_disabled` public input is set. This public input is the value of the instance column in row `orchard::circuit::DISABLE_CROSS_ADDRESS=9`. The verifying key of this circuit is the one that has to be supplied to the verifier calls. Note that if the equality is enforced, the spent part of the action has to be authorized, and thereby the spend authority for the receiving address has to sign.

The public input (instance) derivation matches the circuit update. Via the chain `orchard::bundle::Bundle::to_instances`, calling `orchard::bundle::Action::to_instance`, calling `orchard::circuit::Instance::from_parts`, the `orchard::circuit::Instance` structs for the bundle actions are generated whose `cross_address_disabled` input is the negation of the `cross_address_enabled` flag. For an OrchardNu6_30nward flag, this means that `cross_address_disabled` is false. Then `orchard::circuit::Instance::to_halo2_instance` wires this to the correct public input in row `orchard::circuit::DISABLE_CROSS_ADDRESS=9`.

For the remaining analysis, we separate both verifying paths. In the **individual bundle** verification, `orchard::bundle::Bundle::verify_proof` calls `orchard::bundle::Bundle::to_instances` to derive the instances. It then calls `orchard::circuit::Proof::verify`, which derives the halo2 instances via `orchard::circuit::Instance::to_halo2_instance` and passes them to `halo2_proofs::plonk::verifier::verify_proof`. This then verifies the action statement, including the enforcement of the cross-address restriction.

In the **batch** verification, `::add_bundle` calls `orchard::bundle::Bundle::to_instances` and, via `orchard::circuit::Proof::add_to_batch`, calls `to_halo2_instance`. Then `::validate` calls `halo2_proofs::plonk::verifier::batch::BatchVerifier::finalize`, which verifies all the action statements of all added bundles, enforcing the cross-address restriction.

Both paths contain a hardening check that verifies the circuit version to be Orchard NU6.3 if

`cross_address_disabled` is set. Thus, the verification results in an error if a different key is supplied for an OrchardNu6_30nward bundle, which always sets this flag.

5.2. Version 3 Note Commitment Trapdoor (r_{cm})

The Ironwood upgrade introduces a new derivation for the note commitment trapdoor r_{cm} , defined for version 3 (quantum-recoverable) note plaintexts. The goal is post-quantum commitment binding. The note commitment `NoteCommit` is only computationally binding, so a quantum adversary able to break the underlying group could open a commitment to a different note. Binding r_{cm} to the full note contents through a collision-resistant hash removes this degree of freedom, which is a precondition for the recovery plan under which Ironwood notes are intended to remain spendable after a post-quantum migration.

Specification

The derivation is defined in [ZIP 2005](#) [§](#). For a version 2 (ZIP 212) note, r_{cm} is derived as $r_{cm} = \text{ToScalar}(\text{PRF}_{rseed}^{\text{expand}}([0x05] \parallel \rho))$, binding it only to $rseed$ and ρ . For a version 3 note, r_{cm} instead commits to every field of the note:

$$\text{pre}_{r_{cm}} = [0x0B] \parallel g_d^* \parallel pk_d^* \parallel \text{I2LEOSP}_{64}(v) \parallel \rho \parallel \text{I2LEOSP}_{256}(\psi)$$

$$r_{cm} = \text{ToScalar}^{\text{Orchard}}\left(\text{PRF}_{rseed}^{\text{expand}}(\text{pre}_{r_{cm}})\right)$$

The domain separator `0x0B` distinguishes this use of $\text{PRF}_{rseed}^{\text{expand}}$ from the other Orchard uses of the same PRF. The value of r_{cm} is not enforced on chain. A transaction whose r_{cm} was derived by any other rule is still accepted by consensus, because the network only ever sees the note commitment `cmx`, not r_{cm} itself. Correctness of the derivation is therefore something the recipient must check, and it becomes spendability-critical only once the post-quantum recovery plan is implemented. The intended invariant is that all Ironwood notes follow this derivation, as only Ironwood (version 3) notes are planned to be recoverable.

Implementation

The derivation is implemented in `orchard::note::RandomSeed::rcm_v3` in `src/note.rs`. It hashes the fields with a 64-byte BLAKE2b state personalized with `Zcash_ExpandSeed` in the order

$$rseed \parallel [0x0B] \parallel g_d^* \parallel pk_d^* \parallel \text{I2LEOSP}_{64}(v) \parallel \rho \parallel \text{I2LEOSP}_{256}(\psi)$$

then maps the digest to a scalar via `ToScalar`. The `g_d` and `pk_d` points are serialized with their compressed encodings, `v` as a little-endian u64, and `rho` and `psi` via `to_repr()` on the underlying `pallas::Base`. The reliance on the implementation-specific byte layout of `to_repr()` for `rho` and `psi` is the subject of [Finding 3.1](#) [§](#).

The note version is carried by the `orchard::note::NoteVersion` field of `orchard::note::Note`, signaled by the note plaintext lead byte (0x02 for version 2, 0x03 for version 3). The `orchard::note::Note::rcm` method dispatches on this field, calling `rcm_v2` for version 2 notes and `rcm_v3` for version 3 notes.

We now describe how the recipient of a version 3 note can check that `rcm` was computed correctly. Crucially, `rcm` is never transmitted. The note plaintext (`orchard::note_encryption::NoteEncryptionDomain::note_plaintext_bytes`) carries only the lead byte, the diversifier, the value, `rseed`, and the memo. The recipient re-derives every remaining component from `rseed` and the on-chain `rho`. On trial decryption, `orchard::note_encryption::parse_note_plaintext_without_memo` reads the diversifier, value, and `rseed`, derives `pk_d` from the recipient's own incoming viewing key, and calls `orchard::note::Note::from_parts`. That constructor immediately calls `commitment_inner`, which recomputes `rcm` and derives the note commitment.

The actual check is the commitment equality test in the `zcash_note_encryption` crate. Its `check_note_validity` compares the recomputed extracted note commitment against the `cmx` published on chain in the action, and rejects the note if they differ. Because `rcm` is one of the inputs to `NoteCommit`, a version 3 note whose sender used any derivation other than `rcm_v3` produces a mismatched `cmx` and fails decryption. Correctness of `rcm` is thus enforced transitively through the commitment check rather than by any dedicated comparison. A version 3 `rcm` that decrypts successfully is, by construction, the `rcm` that `rcm_v3` prescribes.

6. Assessment Results

During our assessment on the scoped Zcash Ironwood Upgrade targets, we discovered three findings. No critical issues were found. One finding was of low impact and the other findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.